Software Construction & Development (Comp-370)

Complex Computing Problem (CCP) Submission

Spring Boot | Microservices | Kafka

Threat Intelligence Processing Platform Using Microservices and Kafka

Submitted By:

Atiqad Hayat
B23F0527SE047

Muhammad Bilal Sami
B23S0295SE014

Submitted To:

Malik Nabeel Ahmed Awan

Date: 2 May 2026

Table of Contents

Table of Contents

Abstract 4

1. Introduction 4

2. Problem Statement 5

3. Objectives 5

4. Scope of the Project 6

5. Technologies Used 6

6. System Architecture 7

7. Microservices Description 8

  7.1 Ingestion Service 8

  7.2 Processing Service 9

  7.3 Ranking Service 9

  7.4 Analytics Service 10

8. System Workflow 10

9. Kafka Producer-Consumer Explanation 10

10. REST API Communication 11

11. Database Design 12

12. API Endpoints 13

13. Implementation Details 14

14. Testing and Results 15

15. Team Collaboration 16

16. Challenges Faced 17

17. Sustainable Development Goals Mapping 18

18. Conclusion 18

19. Future Enhancements 19

20. References 19

Appendix: Diagrams and Screenshots 20

Architecture Diagram 20

Sequence Diagram 21

Activity Diagram 22

Testing Results and System Verification 23

7.1 Ingestion Service Testing 24

7.2 Kafka Consumer and Processing Service Testing 24

7.3 MySQL Database Verification 25

7.4 Analytics Service Testing 26

GitHub Repository:
https://github.com/atiqad/threat-intelligence-platform

Abstract

This project is a microservices-based Threat Intelligence Processing Platform. The main purpose of the system is to process Indicators of Compromise, also called IOCs. In this project, IOCs include IP addresses and domain names that may be related to suspicious or malicious activity.

The system uses Apache Kafka for asynchronous communication between services. It also uses REST APIs for synchronous communication where direct service-to-service communication is required. MySQL is used as the persistent database to store processed and ranked IOC data.

The project is implemented using four Spring Boot microservices: Ingestion Service, Processing Service, Ranking Service, and Analytics Service. The Ingestion Service prepares IOC data and sends it to Kafka. The Processing Service consumes the IOC data from Kafka, calls the Ranking Service to get severity, and stores the final data in MySQL. The Analytics Service reads the stored IOC data and provides query results through REST APIs.

This project demonstrates the use of distributed systems, event-driven architecture, REST communication, database integration, and basic cybersecurity data processing.

1. Introduction

Cybersecurity systems often need to process large amounts of threat intelligence data. This data may include suspicious IP addresses, malicious domains, URLs, file hashes, or other security indicators. These indicators are known as Indicators of Compromise, or IOCs.

In real-world cybersecurity platforms, data usually comes from many external sources. This data must be collected, cleaned, processed, ranked, stored, and made available for analysis. A traditional monolithic application is not always suitable for this kind of system because different parts of the system may need to scale independently.

For this reason, this project uses a microservices architecture. Each service has a separate responsibility. Apache Kafka is used to transfer IOC data asynchronously from one service to another. REST APIs are used where direct communication is required, such as when the Processing Service calls the Ranking Service.

The final system provides a working data pipeline where IOC data is ingested, sent through Kafka, processed, ranked, saved in MySQL, and then displayed through an analytics API.

2. Problem Statement

The problem is to design and implement a distributed cybersecurity platform that can process threat intelligence data in a reliable and scalable way.

The system must process IOC data such as IP addresses and domain names. It should support asynchronous communication so that one service does not directly depend on the immediate response of another service. It should also store processed results permanently in a database.

The main challenge is to design the system in a loosely coupled way. If all features are implemented inside one application, the system becomes difficult to maintain and scale. Therefore, this project separates the system into multiple microservices.

The platform must perform the following tasks:

- Ingest IOC data
- Extract IP addresses and domain names
- Send extracted data through Kafka
- Consume and process IOC data
- Rank IOC severity
- Store processed data in MySQL
- Provide analytics/query APIs

3. Objectives

The main objectives of this project are:

- To design a microservices-based cybersecurity platform
- To process IOC data such as IP addresses and domain names
- To use Apache Kafka for asynchronous communication
- To use REST APIs for synchronous service communication
- To store ranked IOC records in MySQL
- To provide analytics APIs for viewing stored IOC data
- To understand event-driven architecture
- To gain practical experience with Spring Boot microservices
- To demonstrate team collaboration using GitHub

4. Scope of the Project

The scope of this project is focused on backend microservices and threat intelligence data processing. The project does not include a full frontend dashboard, but it provides REST APIs that can be tested through a browser or Postman.

The implemented scope includes:

- Four independent Spring Boot microservices
- Kafka producer and consumer communication
- REST API communication between Processing Service and Ranking Service
- MySQL database storage
- Severity ranking logic
- Analytics API for fetching stored records
- GitHub repository for code submission and collaboration

The system currently uses sample IOC data instead of live integration with external threat intelligence APIs. The Ranking Service simulates external ranking API behavior by returning severity values based on IP address patterns.

5. Technologies Used

Java 17

Java 17 was used as the main programming language for all microservices. It provides stability and compatibility with modern Spring Boot applications.

Spring Boot

Spring Boot was used to create REST APIs and microservices quickly. It also simplifies dependency management and application configuration.

Apache Kafka

Apache Kafka was used for asynchronous communication between the Ingestion Service and Processing Service. Kafka helps decouple services and supports event-driven communication.

Docker

Docker was used to run Kafka and Zookeeper easily on the local system.

MySQL

MySQL was used as the persistent database for storing processed IOC data.

REST APIs

REST APIs were used for service communication and for testing endpoints through browser/Postman.

GitHub

GitHub was used for source code management, repository sharing, commits, and team collaboration.

6. System Architecture

The system is divided into four microservices:

1. Ingestion Service
2. Processing Service
3. Ranking Service
4. Analytics Service

Each service has a separate responsibility. This separation helps make the system modular and easier to maintain.

The Ingestion Service sends IOC data to Kafka. The Processing Service consumes the data from Kafka and calls the Ranking Service through REST API. The Ranking Service returns a severity score. The Processing Service stores the final ranked IOC data in MySQL. The Analytics Service reads data from MySQL and provides it through REST APIs.

Architecture Flow

User request → Ingestion Service → Kafka Topic → Processing Service → Ranking Service → MySQL Database → Analytics Service

Communication Types

- Ingestion Service to Kafka: Asynchronous communication
- Kafka to Processing Service: Asynchronous communication
- Processing Service to Ranking Service: Synchronous REST API communication
- Processing Service to MySQL: Database communication
- Analytics Service to MySQL: Database query communication

7. Microservices Description

7.1 Ingestion Service

Port: 8080
Main API: GET `/ingestion/send`

The Ingestion Service is responsible for preparing IOC data. In this implementation, the service provides sample threat data containing IP addresses and domain names. It extracts the important IOC information and sends it to Apache Kafka using a Kafka producer.

Responsibilities

- Provides IOC data
- Extracts IP addresses and domain names
- Sends extracted data to Kafka
- Acts as Kafka Producer

Example IOC Data

- IP Address: 192.168.1.1
- Domain: malicious.com
- IP Address: 10.0.0.5
- Domain: phishing.net

7.2 Processing Service

Port: 8081

The Processing Service consumes IOC messages from Kafka. After receiving the message, it separates the IP address and domain name. It then calls the Ranking Service using REST API to get the severity score. After receiving the severity, it stores the final record in MySQL.

Responsibilities

- Consumes data from Kafka
- Processes IOC records
- Validates and separates IP/domain data
- Calls Ranking Service
- Saves final data into MySQL
- Acts as Kafka Consumer

7.3 Ranking Service

Port: 8083
Main APIs:

- GET `/ranking/test`
- GET `/ranking/score/{ip}`

The Ranking Service provides severity score for IP addresses. In this project, it simulates an external ranking API. It returns severity values such as LOW, MEDIUM, or HIGH based on the IP address pattern.

Responsibilities

- Receives IP address from Processing Service
- Calculates severity score
- Returns LOW, MEDIUM, or HIGH
- Provides REST API for ranking

Severity Logic

- IP starting with `192` → LOW
- IP starting with `10` → MEDIUM
- Other IPs → HIGH

7.4 Analytics Service

Port: 8082
Main APIs:

- GET `/analytics/test`
- GET `/analytics/iocs`

The Analytics Service reads processed IOC data from MySQL. It provides REST APIs to view saved IOC records. This service is useful for querying and analyzing threat intelligence data.

Responsibilities

- Reads stored IOC records from MySQL
- Provides query/analytics API
- Displays processed IOC data

8. System Workflow

The system workflow explains how data moves from one service to another.

Step-by-Step Workflow

1. User sends a request to the Ingestion Service.
2. The Ingestion Service prepares IOC data.
3. The Ingestion Service extracts IP addresses and domain names.
4. The extracted IOC data is published to Kafka topic `ioc-topic`.
5. The Processing Service consumes data from Kafka.
6. The Processing Service separates the IP address and domain.
7. The Processing Service calls the Ranking Service using REST API.
8. The Ranking Service returns severity score.
9. The Processing Service saves the final IOC record in MySQL.
10. The Analytics Service fetches records from MySQL.
11. User can view processed IOC records through the Analytics API.

9. Kafka Producer-Consumer Explanation

Apache Kafka is used in this project to support event-driven communication.

Kafka Topic

The topic used in this project is:

```
ioc-topic
```

This topic carries IOC messages from the Ingestion Service to the Processing Service.

Kafka Producer

The Ingestion Service acts as the Kafka Producer. It publishes extracted IOC data to the Kafka topic.

Example message format:

```
192.168.1.1,malicious.com
```

Kafka Consumer

The Processing Service acts as the Kafka Consumer. It listens to the topic `ioc-topic` and receives IOC messages whenever new data is published.

Why Kafka is Used

Kafka is used because it supports asynchronous communication. This means the Ingestion Service does not need to wait for the Processing Service to be available immediately. The message is published to Kafka, and the Processing Service can consume it from the topic.

This makes the system more loosely coupled and closer to real-world distributed system design.

10. REST API Communication

REST APIs are used for synchronous communication in this project.

The main synchronous communication happens between:

Processing Service → Ranking Service

When the Processing Service receives IOC data, it calls the Ranking Service using this endpoint:

```
GET /ranking/score/{ip}
```

The Ranking Service returns the severity score. The Processing Service then stores the final ranked data in MySQL.

REST API is also used for user testing and analytics.

11. Database Design

The project uses MySQL for persistent storage.

Database Name

`threat_intelligence_db`

Table Name

`iocs`

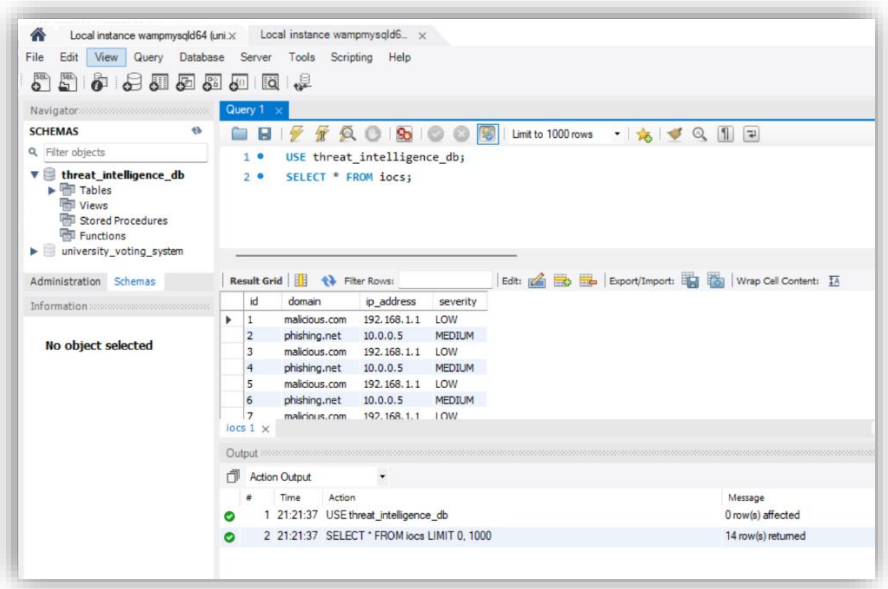
Table Structure

Column Name	Data Type	Description
id	BIGINT	Primary key, auto-generated
ip_address	VARCHAR	Stores IOC IP address
domain	VARCHAR	Stores IOC domain name
severity	VARCHAR	Stores severity score

Sample Stored Data

id	ip_address	domain	severity
1	192.168.1.1	malicious.com	LOW

id	ip_address	domain	severity
2	10.0.0.5	phishing.net	MEDIUM



12. API Endpoints

Ingestion Service APIs

Method	Endpoint	Description
GET	`/ingestion/send`	Sends IOC data to Kafka

Ranking Service APIs

Method	Endpoint	Description
GET	`/ranking/test`	Tests Ranking Service
GET	`/ranking/score/{ip}`	Returns severity score for IP

Analytics Service APIs

Method	Endpoint	Description
GET	`/analytics/test`	Tests Analytics Service
GET	`/analytics/iocs`	Returns all stored IOC records

Important Test URLs

- `http://localhost:8080/ingestion/send`
- `http://localhost:8083/ranking/test`

- `http://localhost:8083/ranking/score/10.0.0.5`
- `http://localhost:8082/analytics/test`
- `http://localhost:8082/analytics/iocs`

13. Implementation Details

Ingestion Service Implementation

The Ingestion Service creates sample IOC data and sends it to Kafka. It uses KafkaTemplate to publish messages to the Kafka topic.

The message contains IP address and domain name separated by a comma.

Processing Service Implementation

The Processing Service uses Kafka Listener to consume messages from the Kafka topic. After consuming the message, it splits the message into IP address and domain.

Then it calls the Ranking Service to get severity score. After receiving the score, it saves the record in MySQL using Spring Data JPA.

Ranking Service Implementation

The Ranking Service provides a simple severity calculation. It accepts an IP address as input and returns severity.

This service simulates external ranking API behavior. In a future version, this service can be connected with real external APIs.

Analytics Service Implementation

The Analytics Service uses Spring Data JPA to fetch IOC records from the MySQL database. It returns the records in JSON format through REST API.

14. Testing and Results

Testing was done using browser and service console logs.

Test 1: Ingestion Service

Endpoint:

`http://localhost:8080/ingestion/send`

Expected Result:

```
Data sent to Kafka
```

[Insert Screenshot of /ingestion/send]

Test 2: Kafka Consumer Console

After calling the Ingestion Service, the Processing Service console shows received Kafka messages.

Expected Console Output:

```
Received from Kafka: 192.168.1.1,malicious.com
```

```
Saved to MySQL: 192.168.1.1 malicious.com LOW
```

```
Received from Kafka: 10.0.0.5,phishing.net
Hibernate: insert into iocs (domain,ip_address,severity) values (?,?,?)
Saved to MySQL: 10.0.0.5 phishing.net MEDIUM
```

Test 3: Ranking Service

Endpoint:

```
http://localhost:8083/ranking/score/10.0.0.5
```

Expected Result:

```
MEDIUM
```

Test 4: Analytics Service

Endpoint:

```
http://localhost:8082/analytics/iocs
```

Expected Result:

The API returns stored IOC records from MySQL in JSON format.

localhost:8082/analytics/iocs

localhost:8082/analytics/iocs

Pretty print


```
[{"domain":"malicious.com","id":1,"ipAddress":"192.168.1.1","severity":"LOW"},{"domain":"phishing.net","id":2,"ipAddress":"10.0.0.5","severity":"MEDIUM"},
{"domain":"malicious.com","id":3,"ipAddress":"192.168.1.1","severity":"LOW"},{"domain":"phishing.net","id":4,"ipAddress":"10.0.0.5","severity":"MEDIUM"},
{"domain":"malicious.com","id":5,"ipAddress":"192.168.1.1","severity":"LOW"},{"domain":"phishing.net","id":6,"ipAddress":"10.0.0.5","severity":"MEDIUM"},
{"domain":"malicious.com","id":7,"ipAddress":"192.168.1.1","severity":"LOW"},{"domain":"phishing.net","id":8,"ipAddress":"10.0.0.5","severity":"MEDIUM"},
{"domain":"malicious.com","id":9,"ipAddress":"192.168.1.1","severity":"LOW"},{"domain":"phishing.net","id":10,"ipAddress":"10.0.0.5","severity":"MEDIUM"},
{"domain":"malicious.com","id":11,"ipAddress":"192.168.1.1","severity":"LOW"},{"domain":"phishing.net","id":12,"ipAddress":"10.0.0.5","severity":"MEDIUM"},
{"domain":"malicious.com","id":13,"ipAddress":"192.168.1.1","severity":"LOW"},{"domain":"phishing.net","id":14,"ipAddress":"10.0.0.5","severity":"MEDIUM"}]
```

15. Team Collaboration

This was a group assignment, so team collaboration was an important part of the project.

Atiqad Hayat

Atiqad Hayat worked on:

- Core microservices development
- Ingestion Service
- Kafka integration
- Processing Service
- Ranking Service
- MySQL integration
- GitHub repository setup
- Testing of main data pipeline

Muhammad Bilal Sami

Muhammad Bilal Sami worked on:

- Technical documentation
- Architecture explanation
- API testing support
- GitHub collaboration
- Report preparation
- Workflow explanation

GitHub Collaboration

GitHub was used to manage the project source code. The repository contains commits and collaboration work from team members. The teammate contributed documentation and project explanation through a separate branch and pull request.

16. Challenges Faced

During the project, several challenges were faced:

Kafka Setup

Kafka setup required Docker and proper configuration. Kafka and Zookeeper had to be started correctly before testing producer and consumer communication.

Microservice Port Management

Each service needed a separate port. The ports used were:

- Ingestion Service: 8080
- Processing Service: 8081
- Analytics Service: 8082
- Ranking Service: 8083

MySQL Connection

MySQL connection required correct database name, username, and password configuration. The database had to be created before running the Processing Service.

Package Structure

In Spring Boot, correct package structure is important. If classes are created outside the correct package, Spring Boot may not detect controllers or repositories.

GitHub Collaboration

GitHub collaboration required proper Git installation, branch creation, commits, push, and pull request creation.

17. Sustainable Development Goals Mapping

This project can be mapped with the following Sustainable Development Goals:

Quality Education

The project supports learning of modern software engineering concepts such as microservices, Kafka, REST APIs, and database integration.

Industry, Innovation and Infrastructure

The project uses industry-relevant technologies and demonstrates a distributed system architecture used in real-world backend systems.

Peace, Justice and Strong Institutions

Cybersecurity systems help protect digital infrastructure and support secure information systems.

Good Health and Wellbeing

Secure systems indirectly support digital safety, privacy, and protection of important services.

18. Conclusion

This project successfully implements a Threat Intelligence Processing Platform using microservices and Kafka. The system processes IOC data such as IP addresses and domain names, sends data through Kafka, ranks the IOC severity, stores results in MySQL, and provides analytics through REST APIs.

The project demonstrates microservices architecture, event-driven communication, REST API integration, and persistent data storage. It also shows how different services can work independently while still forming a complete distributed system.

The final system is a working backend platform that meets the main CCP requirements and provides a strong foundation for future improvements.

19. Future Enhancements

The project can be improved in the future by adding:

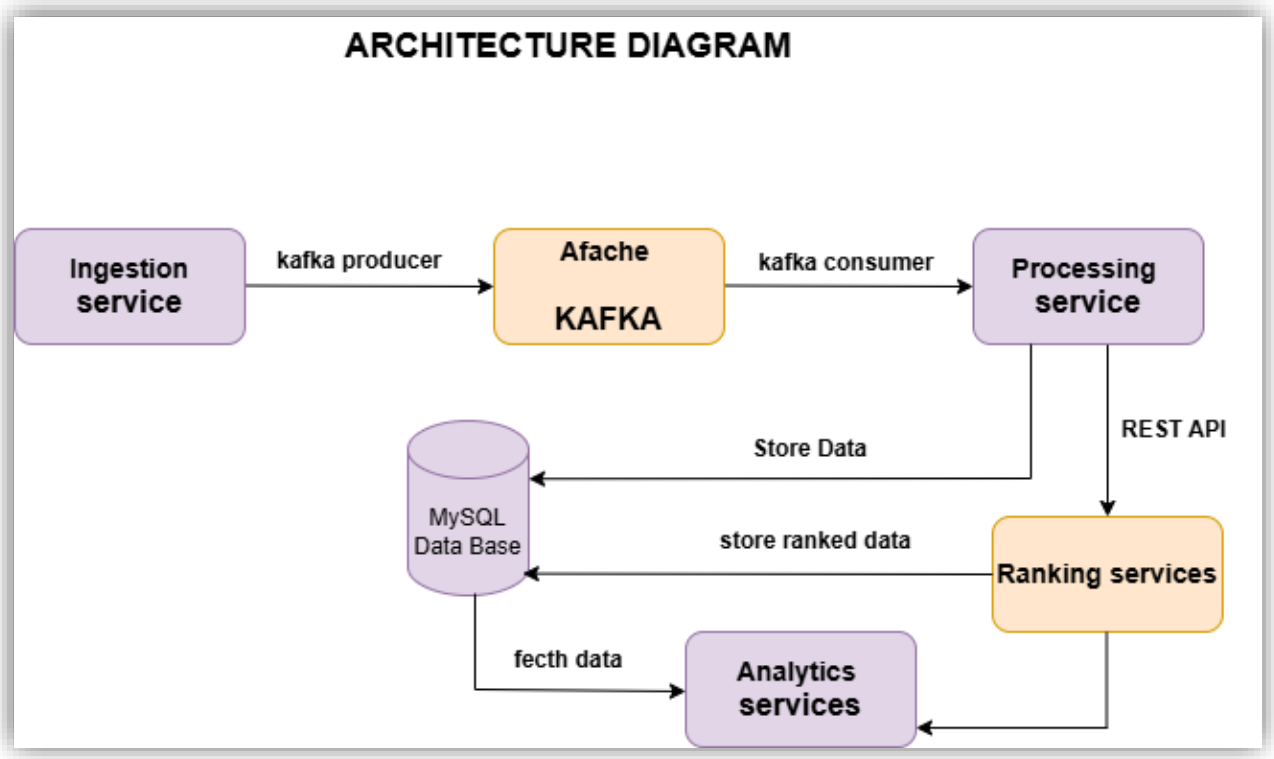
- Real integration with AbuseIPDB API
- Real integration with AlienVault OTX API
- Frontend dashboard for analytics
- Authentication and authorization
- Better severity ranking algorithm
- Duplicate IOC detection
- Error handling and retry mechanism
- API Gateway
- Eureka Service Discovery
- Dockerization of all microservices
- Real-time charts and monitoring

20. References

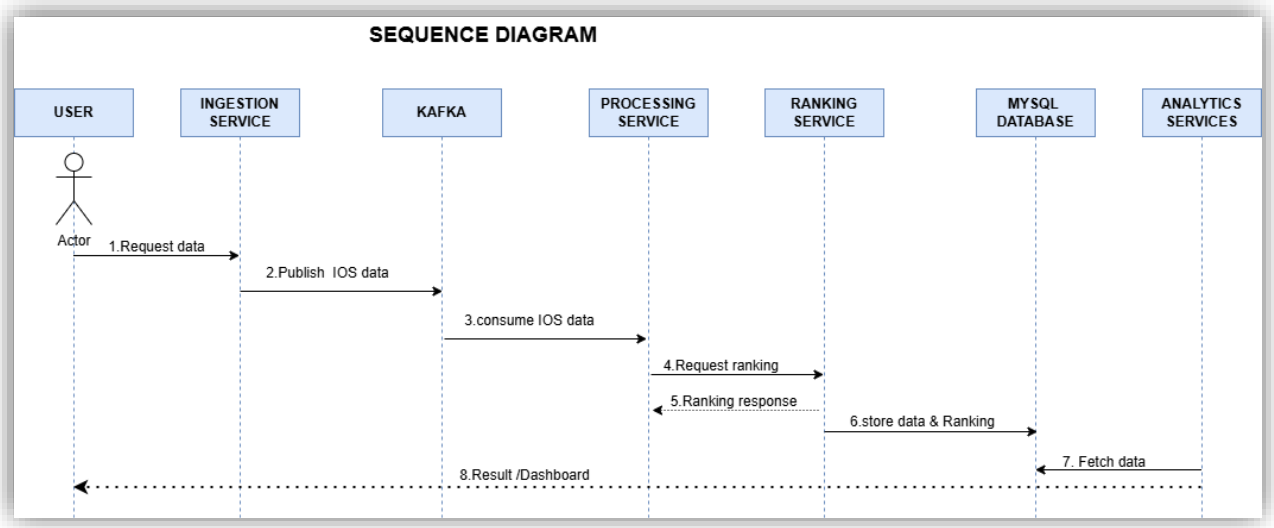
- Apache Kafka Documentation
- Spring Boot Documentation
- MySQL Documentation
- Docker Documentation
- REST API Design Concepts
- Microservices Architecture Concepts

Appendix: Diagrams and Screenshots

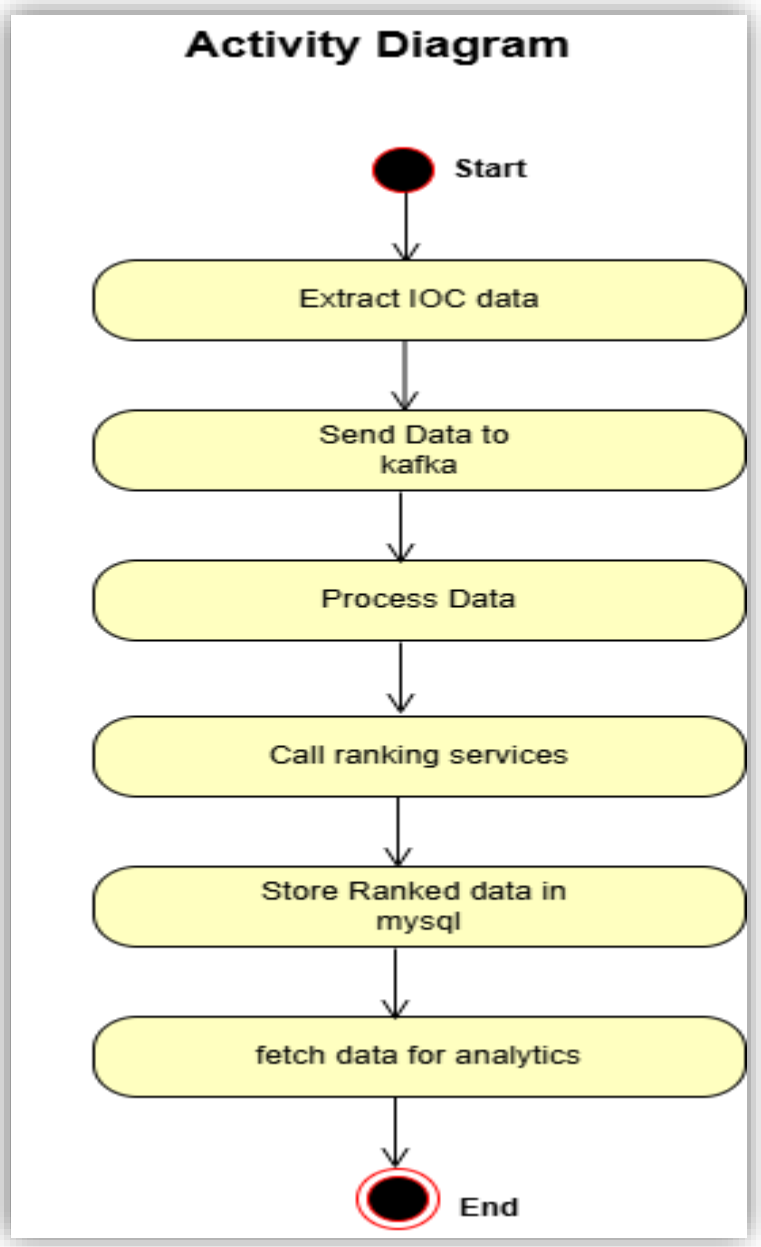
Architecture Diagram



Sequence Diagram



Activity Diagram



Testing Results and System Verification

The following screenshots show the testing results of the implemented system. They verify that data is successfully sent from the Ingestion Service to Kafka, consumed by the Processing Service, ranked using the Ranking Service, stored in MySQL, and displayed through the Analytics Service.

7.1 Ingestion Service Testing

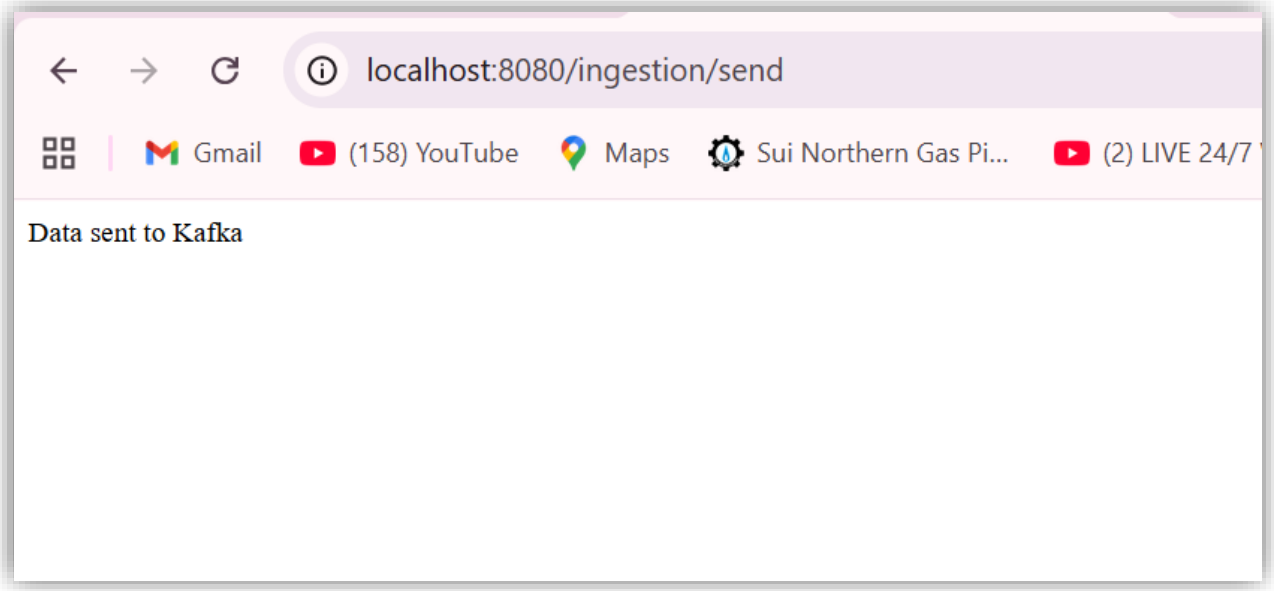


Figure 1: The Ingestion Service successfully sends IOC data to the Kafka topic using the /ingestion/send API.

7.2 Kafka Consumer and Processing Service Testing

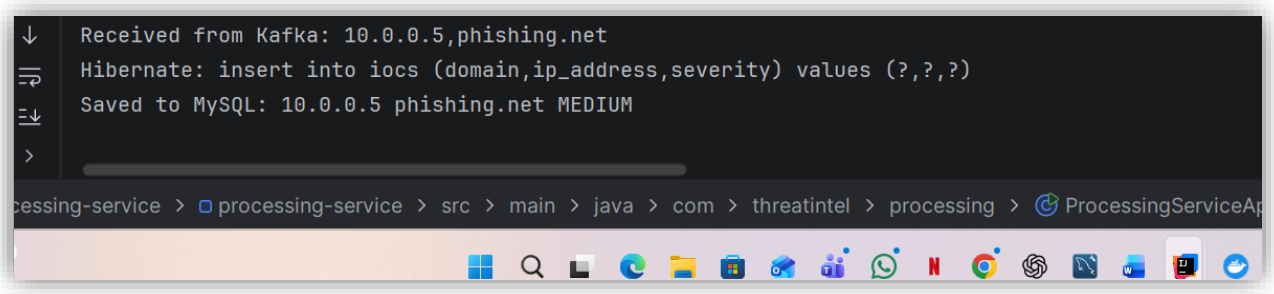


Figure 2: The Processing Service consumes IOC data from Kafka, calls the Ranking Service, and saves the ranked IOC record into MySQL.

7.3 MySQL Database Verification

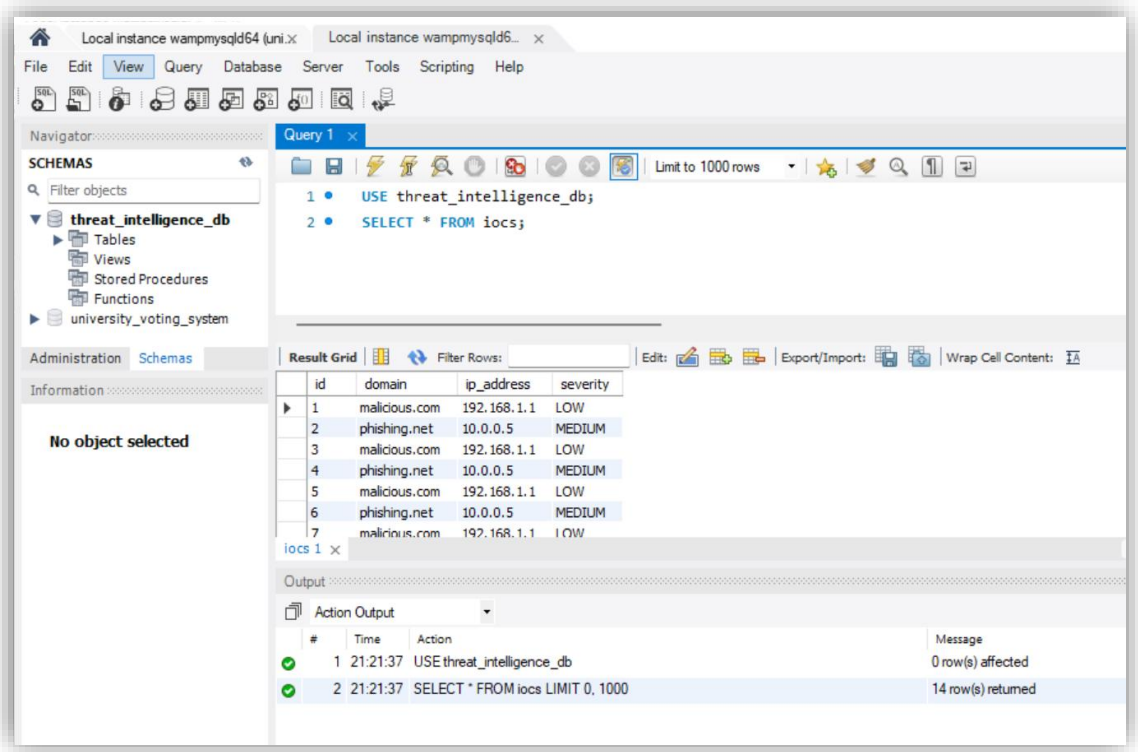


Figure 3: MySQL database table showing stored IOC records with IP address, domain name, and severity score.

7.4 Analytics Service Testing

Figure 4: The Analytics Service retrieves processed IOC records from MySQL using the /analytics/iocs API.